Algorithm II: Predictive Propensity — Technical Strategy Document | FirstKnock Sales OS

Algorithm II: Predictive Propensity

Engineering Specification & Technical Blueprint — FirstKnock Sales OS

2025

CLASSIFICATION: Internal Engineering Blueprint | Audience: FirstKnock Development Team | Version: 2.0

This document constitutes the full engineering specification for Algorithm II: Predictive Propensity. It is intended as the authoritative technical blueprint for the FirstKnock development team and should be read in conjunction with the existing Algorithm I codebase. All formulas, parameter ranges, and pseudocode are production-grade recommendations derived from peer-reviewed PropTech research.

Executive Summary

What Algorithm II Is and Why It Matters

Algorithm I established the foundational scoring architecture for FirstKnock: binary flag detection (absentee ownership, vacancy, high equity) combined with a linear weighted sum, DBSCAN/K-Means clustering, and nearest-neighbor routing with 2-Opt refinement. It works. But it treats field sales as a static targeting problem — identify likely sellers, route to them, knock. Algorithm II reframes the entire system as a unified data-and-execution feedback loop, where every conversion outcome recalibrates the scoring model, every cluster is weighted by revenue density rather than geographic proximity, and every route is engineered around human endurance as a first-class constraint.

The core thesis is this: field sales performance is not primarily a routing problem or a targeting problem in isolation — it is an optimization problem across three coupled systems simultaneously. The quality of the propensity score determines which doors get knocked. The quality of the cluster determines how efficiently those doors are grouped. The quality of the route determines how many of those doors a human being can physically reach before fatigue degrades their conversion rate. Algorithm II engineers all three layers as a single integrated system, with continuous Bayesian recalibration ensuring the model improves with every shift worked.

Key Architectural Upgrades Over Algorithm I

- Continuous Propensity Scoring: Binary flags (absentee_owner === true) replaced by normalized continuous features and composite distress indicators across 800+ signal dimensions

- Ownership duration decay: f_decay(t) = e^(-lambdat) replaces static last_sale_date flag
- Composite distress scoring: D(p) = alpha·equity_gap + beta·vacancy_flag + gamma·absentee_distance + delta·DOM_normalized
- Market momentum features: f_momentum = (current_value - 90d_avg) / 90d_std
- Bayesian Weight Calibration: Static weights replaced by Beta-Binomial conjugate priors updated in real-time from field conversion outcomes via Thompson Sampling
- Parameter update: alpha_new = alpha + conversions, beta_new = beta + non-conversions
- Deconfounded Thompson Sampling (DTS) handles macroeconomic regime shifts (interest rate hikes, seasonal cycles)
- Propensity-Weighted Clustering: Cluster formation integrates propensity scores as sample weights, not as a post-filter — high-value targets gravitationally anchor cluster centroids
- HDBSCAN auto-parameter selection for mixed-density suburban/urban grids
- Revenue density metric R(C) = £ S (p,/)travel_time(C) drives cluster prioritization
- Fatigue-Aware Routing: Human energy depletion modeled as E(t) = E_max · e^(-kappat) + recovery_events, with three intensity zones governing target sequencing across a shift
- Virtual break node injection every 25–30 stops via VRPTW formulation
- Morale optimization: high-propensity targets sequenced post-break for psychological momentum injection

Expected Performance Lift

geq 0.75vs ~0.60 baselineAUC-ROC Target (Production)	**82.33%Research benchmark**Max Predictive Accuracy	**+25–40%Expected Algorithm II lift**Conversions Per Shift	**leq 0.15Calibration quality**Brier Score Target
95%ML model benchmark Valuation Accuracy	**" 1 5 – 2 5%Loop vs line topology**Route Distance Reduction	**" 6 9 %Clustering + fatigue model**Travel Time Reduction	**< 2sleq 60 stops per cluster**Route Generation Latency

The architectural upgrade from Algorithm I to Algorithm II is not incremental — it is a categorical shift from static rule-based targeting to a self-improving probabilistic system. The sections that follow constitute the complete engineering specification for every component of that system.

Section 1: Predictive Data Pipeline Architecture

1.1 RentCast API Signal Extraction Framework

The RentCast API provides access to 140M+ property records. Not all fields carry equal predictive signal. The following analysis ranks fields by their contribution to propensity score accuracy, based on their role in composite distress modeling, behavioral trigger detection, and financial health

assessment. Fields are classified into three tiers: Primary Signal (direct causal relationship with selling propensity), Secondary Signal (contextual enrichment), and Interaction Features (cross-field composites that generate non-linear signal).

Table 1.1 — RentCast Field Signal Priority Ranking and Feature Engineering Mapping

RentCast Field	Engineered Feature Name	Signal Tier	Feature Type	Justification
equity_percent	f_equity	Primary	Continuous [0,1]	High LTV owners set higher asking prices, longer DOM, higher sale prices — direct selling behavior predictor
last_sale_date	f_decay(t)	Primary	Continuous decay	Ownership duration via exponential decay; longer tenure = higher accumulated equity + life event probability
is_vacant	f_vacancy	Primary	Binary rightarrow continuous	Vacancy is the strongest single distress proxy; carrying costs accelerate motivation to sell
absentee_owner	f_absentee	Primary	Binary + distance	Absentee status combined with mailing address distance creates composite motivation signal
owner_mailing_address_distance	f_distance	Primary	Continuous (km)	Distance from property amplifies absentee signal; >50km threshold indicates non-local investor

days_on_market	f_dom_normalized	Primary	Continuous [0,1]	DOM normalized against local median; high relative DOM signals pricing misalignment or distress
assessed_vs_market_gap	f_valuation_gap	Secondary	Continuous ratio	Gap between assessed and market value indicates equity miscalculation or distressed pricing
rental_estimate	f_rental_yield	Secondary	Continuous ($)	Rental yield vs carrying cost ratio; low yield on high-value property signals sell motivation
owner_occupancy_status	f_occupancy	Secondary	Categorical	Owner-occupied properties have lower propensity; inverse weight in composite score
new_construction_permits	f_permit_activity	Secondary	Count/radius	Neighborhood permit activity signals appreciation momentum — contextual enrichment
absorption_rate	f_absorption	Interaction	Continuous [0,1]	Market-level supply/demand ratio; modulates all individual property scores by regime
equity_percent times absentee_flag	f_equity_absentee	Interaction	Cross-product	High equity + absentee = highest-value composite signal for motivated seller identification

The priority ordering above is grounded in behavioral economics research on seller motivation. Equity position (f_equity) is ranked first because owners with higher Loan-to-Value ratios demonstrably alter their pricing and timing behavior — they are not simply more likely to sell, they are more likely to sell at a specific price point and timeline that field reps can exploit. Ownership duration decay (f_decay) is ranked second because survival analysis using the Cox proportional hazards model confirms that time-in-ownership is the single strongest predictor of imminent sale probability, with poor location and larger floor areas further decreasing the odds of a quick sale. Vacancy (f_vacancy) is ranked third because carrying costs on vacant properties create a time-bounded financial pressure that is both measurable and predictable.

Critical Implementation Note: The absentee_owner flag alone (Algorithm I) captures only binary status. Algorithm II requires the composite f_absentee_distance = absentee_flag times log(1 + distance_km), which amplifies the signal for distant non-local owners who face the highest management friction and are therefore most motivated to liquidate.

1.2 Multi-Dimensional Propensity Scoring Model

Algorithm I uses a linear weighted sum of binary flags: Score = (B times W_base) + (V times W_vacant) + (A times W_absentee) + (E times W_equity). This formulation has two fundamental limitations: (1) binary flags discard the magnitude of each signal — a property with 95% equity is treated identically to one with 71% equity; (2) static weights cannot adapt to market regime changes or field-observed conversion patterns. Algorithm II replaces this with a normalized continuous weighted composite:

S(p) = £ bw b f b (p) where i in {decay, equity, vacancy, absentee, momentum, distress, dom, valuation_gap}

Each feature f b (is) normalized to [0, 1] before weighting. The composite score S(p) is then calibrated to a conversion probability via Platt scaling or isotonic regression against historical conversion outcomes. The output is a score in [0, 1] representing the estimated probability of a homeowner listing within 90 days.

Normalization Methodology

Two normalization strategies are applied depending on feature distribution characteristics:

- Min-Max Normalization (bounded features): f_norm = (x - x_min) / (x_max - x_min) — applied to equity_percent, days_on_market, owner_mailing_address_distance. Preserves relative magnitude within known bounds.
- Z-Score Normalization (unbounded features): f_norm = (x - mu) / sigma — applied to assessed_vs_market_gap, rental_estimate, permit_activity_count. Handles outliers without hard clipping.
- Sigmoid Transformation (decay outputs): f_norm = 1 / (1 + e^(-z)) — applied to f_decay(t)

outputs to ensure smooth [0,1] mapping for ownership duration features.

- Clipping Rule: All normalized features are clipped to [0.01, 0.99] to prevent log(0) errors in downstream Bayesian calculations.

Initial Weight Configuration (Pre-Calibration)

Table 1.2 — Initial Feature Weights and Bayesian Prior Configuration

Feature f b	Initial Weight w b	Priority Basis	Bayesian Prior
f_vacancy	0.28	Highest distress signal — Algorithm I validated	Beta(5, 15)
f_equity	0.22	Direct financial motivation — LTV research confirmed	Beta(4, 16)
f_absentee_distance	0.18	Composite absentee + distance amplifier	Beta(3, 17)
f_decay(t)	0.14	Ownership duration survival signal	Beta(3, 17)
f_dom_normalized	0.08	Market timing signal — secondary	Beta(2, 18)
f_momentum	0.05	Market trend context — tertiary	Beta(2, 18)
f_valuation_gap	0.03	Equity miscalculation proxy	Beta(1, 19)
f_occupancy (inverse)	-0.10	Owner-occupied inverse penalty	Beta(2, 18)

The initial weights above are informed priors, not fixed values. The Bayesian calibration engine in Section 1.4 will update these weights continuously as field conversion data accumulates. After approximately 500 field interactions, the posterior distributions will dominate the priors and weights will reflect actual market-specific conversion patterns.

1.3 Feature Engineering Specifications

Ownership Duration Decay Function

The decay function models the increasing probability of sale as ownership duration extends. The mathematical formulation uses exponential decay:

f_decay(t) = e^(-lambdat) where t = years since last_sale_date, lambda = decay constant

This function produces a value approaching 1.0 for recently purchased properties (low sell

probability) and approaching 0.0 for long-held properties (high sell probability). Note: because we want high values to indicate high propensity, the feature is inverted in the scoring pipeline: f_decay_propensity(t) = 1 - e^(-lambdat).

Table 1.3 — Decay Constant lambda Parameter Selection Guide

lambda Value	Decay Behavior	Recommended Context	t=5yr Output	t=10yr Output	t=20yr Output
0.10	Slow decay — long ownership cycles	Rural markets, stable suburban areas	0.394	0.632	0.865
0.15	Moderate decay — balanced markets	General suburban deployment (recommended default)	0.528	0.777	0.950
0.20	Fast decay — high-turnover markets	Urban cores, investor-heavy markets	0.632	0.865	0.982
0.25	Aggressive decay — rapid cycle markets	Hot metro markets, high appreciation zones	0.713	0.918	0.993

Recommended implementation: lambda = 0.15 as the system default, with market-regime-based adjustment: lambda_hot = 0.20, lambda_neutral = 0.15, lambda_cold = 0.10. The regime is detected by the Hidden Markov Model described in Section 1.4.

Market Trend Momentum Feature

Market momentum captures whether a property's local market is appreciating or depreciating relative to its recent baseline. A rising market increases seller motivation (lock in gains); a falling market may trigger distress selling.

f_momentum = (current_value - mu_90d) / sigma_90d where mu_90d = 90-day rolling mean of local median price, sigma_90d = 90-day rolling standard deviation

This is a z-score of current market value against the 90-day baseline. Positive values indicate appreciation momentum (moderate positive weight in scoring). Negative values indicate depreciation (strong positive weight — distress signal). The feature is clipped to [-3, +3] standard deviations to prevent outlier dominance, then normalized to [0, 1] via min-max over the clipped range.

Neighborhood context features are computed within a configurable radius (default: 0.5km urban, 1.5km suburban, 3.0km rural) around each target property. These features capture the local market environment that modulates individual property propensity:

- f_absorption_rate: Absorption rate = (properties sold in 30d) / (total active listings). Thresholds: <15% = Buyer's Market (dampens propensity), 15–20% = Balanced, >20% = Seller's Market (amplifies propensity)
- f_vacancy_density: Proportion of vacant properties within radius. High local vacancy density amplifies individual vacancy signal — indicates neighborhood-level distress
- f_equity_median: Median equity ratio of neighboring properties. Low neighborhood equity signals systemic financial pressure — amplifies individual distress composite
- f_permit_velocity: Count of new construction/renovation permits filed in last 90 days within radius. High permit activity signals appreciation momentum — positive contextual modifier
- f_dom_local: Local median days on market. Individual property DOM normalized against this local baseline rather than national average — prevents geographic bias

Distress Signal Composite

The distress composite D(p) aggregates the four strongest individual distress signals into a single normalized indicator. This composite is used both as a standalone feature in S(p) and as a multiplier on the base propensity score for properties exceeding the distress threshold:

D(p) = alpha · f_equity_gap + beta · f_vacancy + gamma · f_absentee_distance + delta · f_dom_normalized

Where f_equity_gap = max(0, 0.70 - equity_percent) / 0.70 (normalized shortfall below 70% equity threshold). Recommended parameter ranges: alpha = 0.35, beta = 0.30, gamma = 0.20, delta = 0.15. These sum to 1.0, making D(p) a properly weighted sub-composite in [0, 1]. Properties with D(p) > 0.65 are flagged as 'High Distress' and receive a score multiplier of 1.15 in the final S(p) calculation, capped at S(p) = 0.99.

Life Event Proxy Features

Advanced propensity systems stack life event proxies to identify owners who genuinely need to sell quickly. While direct life event data is not available via RentCast, the following proxy signals can be engineered from available fields and third-party enrichment:

- Divorce proxy: Property ownership change within 2 years of purchase (rapid resale pattern) — detectable via last_sale_date + prior_sale_date delta
- Job relocation proxy: Absentee flag onset within 6 months of last_sale_date — owner moved before selling, indicating relocation pressure
- Estate/probate proxy: Owner age > 75 (where available via public records enrichment) combined with long ownership duration (t > 20 years)

- Financial distress proxy: D(p) > 0.70 combined with f_dom_normalized > 0.80 — property has been sitting with high distress indicators

1.4 Dynamic Weight Calibration Engine

Bayesian Updating Framework

The Bayesian weight calibration engine treats each feature weight configuration as a hypothesis and updates its probability given observed field conversion outcomes. The mathematical foundation is Bayes' Theorem:

P(theta | data) " P(data | theta) times P(theta) [Posterior " Likelihood times Prior]

For binary conversion outcomes (homeowner lists = 1, does not list = 0), the Beta-Binomial conjugate model provides closed-form parameter updates without numerical integration. The Beta distribution Beta(alpha, beta) serves as the conjugate prior for the Binomial likelihood:

alpha_new = alpha_prior + conversions_observed | beta_new = beta_prior + non_conversions_observed

The posterior mean conversion probability for a given weight configuration is: mu_posterior = alpha_new / (alpha_new + beta_new). The posterior variance (confidence interval width) is: sigma²_posterior = (alpha_new times beta_new) / ((alpha_new + beta_new)² times (alpha_new + beta_new + 1)).

Concrete example: A weight configuration with prior Beta(3, 17) (reflecting 15% baseline conversion rate) observes 720 conversions and 3,280 non-conversions from field data. The posterior updates to Beta(723, 3297), yielding mu_posterior = 723/4020 = 18.0% — a measurable lift over the 15% prior, with dramatically reduced uncertainty due to the large sample.

Thompson Sampling for Live A/B Weight Testing

Thompson Sampling implements a Multi-Armed Bandit (MAB) framework to continuously test alternative weight configurations against the current production weights. Each weight configuration variant k maintains its own Beta(alpha_k, beta_k) distribution. At each routing assignment, the system:

1. Samples a conversion probability theta_k ~ Beta(alpha_k, beta_k) for each active weight variant
2. Assigns the current rep session to the variant with the highest sampled theta_k (exploitation of best-known weights)
3. With probability epsilon = 0.10, overrides to a random variant (exploration of untested configurations)
4. After session completion, updates the winning variant's Beta parameters with observed conversion outcomes
5. Variants with posterior mean < (best_variant_mean - 0.05) for 500+ observations are retired

Deconfounded Thompson Sampling (DTS) is applied when macroeconomic regime shifts are

detected (e.g., Federal Reserve rate announcements, seasonal transitions). DTS uses inverse propensity weights to project population-level performance, preventing the algorithm from incorrectly attributing macro-driven conversion changes to weight configuration quality.

Market Condition Sensitivity Matrix

Market regime is detected using a Hidden Markov Model (HMM) with three latent states: Hot, Neutral, Cold. Observable emissions are absorption_rate, median_DOM, and list-to-sale ratio. The HMM is trained on MSA-level historical data and updates weekly. Once a regime is detected, feature weights are conditionally adjusted per the following matrix:

Table 1.4 — Market Condition Weight Sensitivity Matrix (≠ additive adjustment to baseline)

Feature	Hot Market Weight ≠	Neutral Market Weight	Cold Market Weight ≠	Rationale
f_vacancy	+0.05	0.28 (baseline)	-0.03	Vacancy urgency amplified in cold markets; less critical in hot (anything sells)
f_equity	+0.03	0.22 (baseline)	+0.05	Equity motivation stronger in cold markets — owners need to sell, not just want
f_momentum	+0.08	0.05 (baseline)	-0.03	Momentum signal most valuable in hot markets for timing optimization
f_decay(t)	0.00	0.14 (baseline)	+0.04	Long-tenure owners more motivated in cold markets — accumulated pressure
f_dom_normalized	-0.02	0.08 (baseline)	+0.04	High DOM more distress-indicative in cold markets than hot
f_absorption_rate	+0.06	N/A (contextual)	-0.04	Absorption rate modulates all weights as a regime multiplier

Spring Season Dampener	Apply times1.15	Apply times1.00	Apply times0.85	Seasonal adjustment for spring selling season signal strength

Drift Detection and Recalibration Triggers

Two classes of drift require monitoring: Feature Drift (input distribution shifts) and Concept Drift (feature-to-target relationship degradation).

- Feature Drift Detection: Population Stability Index (PSI) computed weekly for each input feature. PSI = £(Actual% - Expected%) times ln(Actual% / Expected%). Threshold: PSI > 0.20 triggers alert; PSI > 0.25 triggers automatic retraining queue
- Concept Drift Detection: ADWIN (Adaptive Windowing) algorithm monitors the rolling AUC-ROC on recent predictions. When ADWIN detects a statistically significant change in the AUC distribution, it flags concept drift. Page-Hinkley test provides secondary confirmation
- KL Divergence Threshold: KL(P_current || P_baseline) > 0.15 on the score distribution triggers immediate recalibration review. KL > 0.30 triggers automatic rollback to last validated checkpoint
- Performance-Based Triggers: AUC drops below 0.70 on a 7-day rolling window rightarrow immediate retraining. Brier score exceeds 0.20 rightarrow weight recalibration. Conversion rate drops >15% vs 30-day baseline rightarrow regime change investigation

Conversion Feedback Loop Architecture

The feedback loop is the critical architectural component that transforms Algorithm II from a static model into a self-improving system. The pipeline operates as follows: (1) Rep logs conversion outcome in mobile app (knocked rightarrow answered rightarrow interested rightarrow converted/rejected); (2) Feedback service receives event and writes to conversion_events table with property_id, rep_id, timestamp, outcome, and active_weight_variant_id; (3) Bayesian update job runs every 4 hours, consuming new conversion events and updating Beta(alpha, beta) parameters for each active weight variant; (4) Score engine re-scores affected properties using updated weights; (5) Cluster assignments are flagged for refresh if >10% of cluster properties have score delta >0.15.

1.5 Statistical Validation Framework

Holdout Set Design — Critical Requirement

MANDATORY: Real estate propensity models must use temporal holdouts — training on past data, testing on future data. Random holdouts are strictly prohibited because they allow future information to leak into training, producing artificially inflated AUC scores that do not reflect

real-world deployment performance. The chronological nature of property sales makes temporal ordering a fundamental data integrity constraint.

Recommended holdout design: Train on months 1–10, validate on month 11, test on month 12. For initial deployment with limited historical data, use a 70/15/15 temporal split. K-fold cross-validation (k=5) is applied within the training window only, using time-series-aware folds (each fold's test set is always chronologically after its training set).

Evaluation Metrics Suite

Table 1.5 — Statistical Validation Metrics, Targets, and Failure Actions

Metric	Target Threshold	Application in FirstKnock Context	Failure Action
AUC-ROC	geq 0.75 (production), geq 0.70 (staging)	Overall ranking ability — can the model rank true converters above non-converters?	Block deployment; trigger retraining
Precision-Recall AUC	geq 0.25 (5times random baseline)	Primary metric for imbalanced data (1–5% conversion rate); focuses on positive prediction accuracy	Investigate class imbalance handling
Brier Score	leq 0.15	Probabilistic calibration — do predicted probabilities match actual conversion rates?	Apply Platt scaling recalibration
KS Statistic	geq 0.30	Separation between converter and non-converter score distributions	Review feature engineering quality
Lift at Decile 1	geq 4.0times	Top 10% of scored properties should convert at 4times the baseline rate	Investigate top-score feature composition
Lift at Decile 2	geq 2.5times	Top 20% should convert at 2.5times baseline — validates mid-tier scoring	Review weight calibration
Calibration Plot R^2	geq 0.90	Reliability diagram — predicted probabilities align with actual rates across deciles	Apply isotonic regression recalibration

The MLOps pipeline for continuous model refinement follows the architecture implemented by industry leaders (Realtor.com, PropPulse AI) using open-source tooling:

1. Nightly Batch Job (02:00 local): Re-score all properties with updated Bayesian weights. Compute PSI for all features against 30-day baseline. Log score distribution statistics to monitoring dashboard
2. Weekly Validation Job (Sunday 04:00): Run full evaluation metrics suite on 7-day holdout. Generate calibration plots. Compare AUC against 4-week rolling baseline. Flag any metric below threshold for engineering review
3. Drift Response Job (event-triggered): Activated by PSI > 0.20 or ADWIN alert. Runs feature importance analysis to identify drifting features. Generates retraining recommendation with estimated impact
4. Monthly Full Retraining: Complete model retraining on rolling 12-month window. XGBoost hyperparameter re-optimization via Bayesian search. Full validation suite before promotion to production
5. Tooling Stack: Evidently (drift monitoring + data quality), Metaflow (pipeline orchestration), MLflow (experiment tracking), scikit-learn (model training), XGBoost (gradient boosting engine)

Section 2: Hybrid Clustering & Revenue Density Optimization

2.1 Algorithmic Decision Logic

Algorithm I uses a binary DBSCAN/K-Means selector based on a single density threshold. Algorithm II implements a three-tier density classifier with HDBSCAN as the intermediate option for mixed-density suburban environments — the most common deployment context for FirstKnock. The decision logic is formalized as follows:

DENSITY THRESHOLDS: URBAN_THRESHOLD = 150 properties/km² | SUBURBAN_THRESHOLD = 30 properties/km² | RURAL_THRESHOLD = <30 properties/km²

Pseudocode — Clustering Algorithm Selector:

function selectClusteringAlgorithm(properties, bbox):density = len(properties) / bbox.area_km2 propensity_variance = computeVariance([p.score for p in properties])if density > URBAN_THRESHOLD: # >150/km² k = optimalK_silhouette(properties, k_range=[5, 25])return KMeans_Weighted(k=k, sample_weight=propensity_scores)elif density > SUBURBAN_THRESHOLD: # 30–150/km² min_cluster_size = max(8, int(len(properties) * 0.03)) return HDBSCAN(min_cluster_size=min_cluster_size,metric='haversine', cluster_selection_method='eom')else: # <30/km² — rural/sparseeps = adaptiveEpsilon(properties, k=3)return DBSCAN(eps=eps, min_samples=3, metric='haversine')function

adaptiveEpsilon(properties, k):distances = kNearestNeighborDistances(properties, k=k) sorted_distances = sort(distances, ascending=True)eps = elbowDetection(sorted_distances) # k-distance graph elbowreturn eps * terrain_multiplier(properties.region_type)

Threshold Calibration Methodology

The density thresholds (150/km² urban, 30/km² suburban) are derived from empirical PropTech deployment data and should be validated against FirstKnock's specific market portfolio. Calibration procedure: (1) Sample 20 representative deployment zones across the target market; (2) Run all three algorithms on each zone; (3) Compute silhouette score and intra-cluster propensity variance for each; (4) Identify the density range where HDBSCAN outperforms both alternatives — this defines the SUBURBAN_THRESHOLD boundaries; (5) Recalibrate thresholds quarterly as new markets are added.

2.2 DBSCAN Parameter Optimization

Epsilon (epsilon) Calculation via K-Distance Graph

The optimal epsilon is determined algorithmically using the k-distance graph elbow detection method. For each property point, compute the distance to its k-th nearest neighbor (where k = min_samples). Sort these distances in ascending order and plot them. The optimal epsilon corresponds to the 'elbow' — the point of maximum curvature where distances sharply increase, indicating the natural boundary between cluster members and noise points.

Elbow Detection Algorithm: Use the Kneedle algorithm (knee_point_detection library) to programmatically identify the elbow. The Kneedle algorithm normalizes the curve and finds the point of maximum distance from the line connecting the first and last points. This eliminates subjective visual inspection.

Adaptive Epsilon Formula: epsilon(region) = epsilon_base times terrain_multiplier times density_adjustment

Table 2.2 — DBSCAN Parameter Configuration by Region Type

Region Type	Terrain Multiplier	min_samples	Typical epsilon Range (km)	Rationale
Dense Urban Core	0.7times	8	0.08–0.15	Tight clusters needed; large eps creates unmanageable mega-clusters

Standard Suburban	1.0times	5	0.20–0.40	Baseline configuration; balanced noise exclusion vs cluster fragmentation
Sparse Suburban	1.4times	4	0.40–0.70	Larger radius needed to avoid over-fragmentation in lower-density areas
Rural / Exurban	2.0times	3	0.70–1.50	Minimum viable cluster formation; noise tolerance must be high

Noise Point Handling

DBSCAN noise points (properties that do not belong to any cluster) require explicit handling rather than silent discard. Two strategies are applied based on propensity score:

- High-Propensity Noise (S(p) > 0.70): Reassign to nearest cluster centroid using Euclidean distance. These are isolated high-value targets that should not be abandoned simply because they lack geographic neighbors
- Low-Propensity Noise (S(p) leq 0.70): Flag as 'Isolated Target' in the database. Add to a secondary route list for opportunistic knocking when a rep is in the vicinity. Do not include in primary cluster assignments
- Noise Ratio Monitoring: If noise_points / total_points > 0.25, the epsilon parameter is too small. Trigger adaptive epsilon recalculation with a 15% upward adjustment

2.3 K-Means Optimization

Optimal K Selection — Dual Validation Method

K selection uses a dual-validation approach combining the Elbow Method (WCSS) with Silhouette Score maximization. Neither method alone is sufficient: the elbow method can be ambiguous, and silhouette scores can favor artificially small K values.

Silhouette Score Formula: s(i) = [b(i) - a(i)] / max(a(i), b(i))

Where a(i) = mean intra-cluster distance for point i, b(i) = mean distance to nearest neighboring cluster for point i. The optimal K maximizes the average silhouette score across all points. Target: s_avg geq 0.35 for cluster acceptance.

K Selection Pseudocode:

function optimalK_silhouette(properties, k_range):scores = {}for k in k_range:kmeans =

KMeans(n_clusters=k, init='k-means++', n_init=10)labels = kmeans.fit_predict(properties.coordinates, sample_weight=properties.propensity_scores)if len(set(labels)) > 1: # Require at least 2 clusters scores[k] = silhouette_score(properties.coordinates, labels,metric='haversine')# Find K with maximum silhouette scoreoptimal_k = max(scores, key=scores.get)# Validate against cluster size constraint (40-60 stops)while (len(properties) / optimal_k) > 60:optimal_k += 1while (len(properties) / optimal_k) < 40 and optimal_k > k_range[0]:optimal_k -= 1return optimal_k

K-Means++ Initialization and Convergence

K-Means++ uses a D² weighting algorithm for centroid initialization: the first centroid is chosen uniformly at random; each subsequent centroid is chosen with probability proportional to the squared distance from the nearest existing centroid. This initialization strategy dramatically reduces the risk of poor local optima and speeds convergence by 2–5times compared to random initialization.

- Convergence criteria: ” c e n t r o i d _ p s 0.001 degrees OR max_iterations = 300 (whichever occurs first)
- n_init = 10: Run K-Means++ 10 times with different random seeds; retain the solution with lowest inertia
- Mini-Batch K-Means: For datasets >50,000 properties in a single deployment zone, switch to MiniBatchKMeans with batch_size = 1024 to reduce computation time while maintaining solution quality within 2% of full K-Means

2.4 Weighted Clustering Architecture

The critical architectural distinction between Algorithm I and Algorithm II clustering is that propensity scores are integrated into cluster formation — not applied as a post-filter. In Algorithm I, clusters are formed geographically and then filtered by score. In Algorithm II, high-propensity properties gravitationally anchor cluster centroids, ensuring that every cluster is organized around its highest-value targets.

Propensity-Weighted K-Means Formulation

Weighted K-Means Objective: minimize £ b£,| i n Cwb,|· | | x,| m u b | | ²

Where w,|= S (p,|)propensity score of property j, x,|= geographic coordinates of property j, m u b = weighted centroid of cluster i. Implementation: sklearn KMeans accepts sample_weight parameter directly. Pass propensity_scores as sample_weight to weight the centroid calculation and the distance-to-centroid objective.

DBSCAN Density Weighting for Propensity Integration

DBSCAN does not natively support sample weights. The workaround is point replication by score quantile: properties in the top propensity quartile (Q4, S(p) > 0.75) are replicated 3times in the input dataset; Q3 (0.50–0.75) replicated 2times; Q2 (0.25–0.50) replicated 1times (no change); Q1

(<0.25) replicated 0.5times (50% random subsample). This creates artificial density around high-propensity properties, causing DBSCAN to form clusters anchored to high-value targets. After clustering, the replicated points are removed and original property assignments are retained.

2.5 Revenue Density Maximization Framework

Revenue density is the primary cluster quality metric for field sales optimization. A geographically tight cluster with low propensity scores is less valuable than a slightly larger cluster with high propensity density. The revenue density framework formalizes this tradeoff:

Cluster Revenue Score: R(C) = £,| i n S (p,| travel_time(C) [propensity-weighted yield per unit time]

Revenue Density: rho(C) = £,| i n S (p,| area(C) [propensity per km² — geographic concentration metric]

Daily Assignment Optimization: maximize £ bR (C subject to: £ b| C bdq rep_daily_capacity (40–60 stops), £ bt r a v e l _ t i mleq(shift_duration (6 hours active), each rep assigned to contiguous clusters only

Cluster prioritization: Rank all clusters by R(C) in descending order. Assign field reps to the top quartile of clusters first. Clusters in the bottom quartile (R(C) < 25th percentile) are held in reserve for secondary deployment or dropped from the daily assignment pool.

P-Median and Capacitated Clustering

For territory design at the weekly/monthly planning horizon (as opposed to daily routing), the p-median model is applied. The p-median problem selects p facility locations (cluster centers) to minimize the total weighted distance from all demand points to their nearest facility. In FirstKnock's context, 'facilities' are rep home bases or staging areas, and 'demand points' are target properties weighted by propensity score. Capacitated clustering adds the constraint that each cluster cannot exceed the rep's daily capacity (40–60 stops), ensuring equitable workload distribution across the field team.

Contiguous Zone Design via REDCAP

The REDCAP (Regionalization with Dynamically Constrained Agglomerative Clustering and Partitioning) algorithm enforces geographic contiguity in cluster assignments — ensuring that each rep's territory forms a single connected region rather than a collection of disconnected patches. REDCAP is enhanced with multi-valued decision diagrams (MDDs) for exact dynamic programming, optimally removing edges from a spatially contiguous tree to produce higher-quality partitions. Implementation: PySAL library provides REDCAP implementation. Apply after initial K-Means/HDBSCAN clustering to enforce contiguity constraints.

2.6 Cluster Quality Validation

Table 2.6 — Cluster Quality Validation Metrics and Acceptance Thresholds

Metric	Formula / Method	Acceptance Threshold	Failure Action
Silhouette Score	s(i) = [b(i)-a(i)] / max(a(i),b(i)), averaged across all points	geq 0.35	Increase K (K-Means) or reduce eps (DBSCAN); re-cluster
Davies-Bouldin Index	Average similarity ratio of each cluster with its most similar cluster	leq 1.5	Investigate cluster overlap; consider HDBSCAN for mixed-density zones
Calinski-Harabasz Score	Between-cluster dispersion / within-cluster dispersion ratio	Maximize (no fixed threshold)	Track trend; declining score indicates cluster quality degradation
Intra-Cluster Propensity CV	CV = sigma(S(p)) / mu(S(p)) within each cluster	leq 0.40	Re-run weighted clustering with higher propensity weight emphasis
Minimum Cluster Propensity Density	Mean S(p) across all properties in cluster	geq 0.45	Merge low-density clusters or reassign to secondary deployment pool
Cluster Size Constraint	Count of properties per cluster	40–60 stops	Split oversized clusters; merge undersized clusters with neighbors
Dunn Index	Min inter-cluster distance / max intra-cluster diameter	Maximize (track trend)	Low Dunn Index indicates poorly separated or non-compact clusters

Section 3: Field Execution System & Human-Centered Routing

3.1 'Loops, Not Lines' Mathematical Formulation

Algorithm I uses nearest-neighbor construction with 2-Opt refinement but does not enforce loop topology. The result is often a linear or near-linear route that requires a dead-head return to the starting point — wasted distance that carries zero conversion value. Algorithm II formalizes the Hamiltonian cycle constraint as a first-class routing requirement.

TSP Hamiltonian Cycle Formulation: minimize £ bd (v bv,b Š s) subject to: v_start = v_end (loop closure), each property visited exactly once, total_distance leq max_shift_distance

The mathematical case for loops over lines is grounded in the Christofides algorithm guarantee: for

Hamiltonian cycles, the tour length is guaranteed to be less than 3/2 (1.5times) the optimal solution. For linear paths with two specified endpoints, performance degrades to up to 5/3 (1.67times) the optimal length — a 11% efficiency penalty that compounds across a full shift.

Loop Efficiency Gain Formula: E_loop = (D_line - D_loop) / D_line

Expected E_loop = 0.15–0.25 (15–25% distance reduction) based on empirical TSP benchmarks for geographic routing problems. The efficiency gain is highest in dense urban clusters (where dead-head distances are proportionally larger) and lowest in sparse rural routes (where the loop closure adds minimal overhead).

Why Loops Enable Superior System Integration

- Dead-head elimination: The return trip from last stop to vehicle/start point is absorbed into the loop topology — every step of the route is productive
- Natural break insertion: Break nodes can be inserted at any point in a loop without creating route discontinuities; in a linear route, break insertion near the midpoint creates two disconnected sub-routes
- Mid-route cluster switching: If a rep completes a cluster early, the loop topology allows seamless transition to an adjacent cluster without backtracking
- Fatigue zone alignment: The loop naturally returns the rep to their vehicle at peak fatigue (end of Zone 3), eliminating the energy cost of a long return walk

3.2 2-Opt Refinement Algorithm

The 2-Opt algorithm is retained from Algorithm I but is now applied to a loop-constrained TSP formulation rather than an open path. The algorithm eliminates route crossings by iteratively testing edge swaps and accepting improvements.

2-Opt Mathematical Formulation: A 2-opt move removes edges (t1, t2) and (t3, t4) and reconnects as (t1, t3) and (t2, t4). The swap is accepted if: d(t1,t3) + d(t2,t4) < d(t1,t2) + d(t3,t4). For symmetric TSP, the length delta calculation requires only O(1) operations.

Full 2-Opt Pseudocode:

function twoOpt(route):improved = trueiteration_count = 0best_distance = routeDistance(route) while improved and iteration_count < MAX_ITERATIONS: # MAX_ITERATIONS = 1000improved = falseiteration_count += 1for i in range(1, len(route) - 1):for j in range(i + 1, len(route)):# O(1) length delta for symmetric TSPdelta = (distance(route[i-1], route[j]) +distance(route[i], route[j+1]) - distance(route[i-1], route[i]) -distance(route[j], route[j+1]))if delta < -CONVERGENCE_THRESHOLD: # CONVERGENCE_THRESHOLD = 0.001# Reverse the segment between i and jroute[i:j+1] = reversed(route[i:j+1])best_distance += deltaimproved = true break # Restart from beginning after improvementif improved:break# Enforce loop closureif route[0] != route[-1]:route.append(route[0])return route# Or-Opt fallback for large routes (n > 200) function orOpt(route, segment_size=3):# Relocate segments of 1, 2, or 3 consecutive stops# More efficient than 2-Opt for large n# O(n^2) per pass vs O(n^2) for 2-Opt but with smaller constant factor...

Time complexity: O(n^2) per iteration, O(n^3) worst case for full convergence. For clusters of 40–60

stops (the target range), 2-Opt converges in <100ms on modern hardware. For n > 200 (large urban clusters), switch to Or-Opt (segment relocation) which achieves comparable solution quality with lower computational overhead.

3.3 Virtual Node Injection Methodology

Virtual break nodes are modeled using the Vehicle Routing Problem with Time Windows (VRPTW) framework. Break nodes are treated as mandatory waypoints with zero conversion value, a fixed service duration of 15 minutes, and a time window constraint that enforces the break within a plusminus5 minute window of the target break time.

Break Node Properties: duration = 15 min | conversion_value = 0 | location = nearest accessible rest point (parking lot, shaded area, convenience store) | time_window = [target_break_time - 5min, target_break_time + 5min]

Break Node Injection Pseudocode:

function injectBreakNodes(route, interval=27, break_duration=15):"""Inject mandatory break nodes every 'interval' stops.Default interval=27 targets the midpoint of the 25-30 stop range."""break_positions = list(range(interval, len(route), interval))offset = 0 # Track position shift as nodes are insertedfor pos in break_positions:adjusted_pos = pos + offset# Find nearest accessible break location to current route positionbreak_location = findNearestBreakPoint(route[adjusted_pos].coordinates,search_radius_m=200,preferred_types=['parking_lot', 'park', 'convenience_store'])break_node = BreakNode(location=break_location, duration_min=break_duration,conversion_value=0,node_type='MANDATORY_BREAK', time_window=computeBreakTimeWindow(route, adjusted_pos))route.insert(adjusted_pos, break_node)offset += 1 # Each insertion shifts subsequent positions by 1return routefunction computeBreakTimeWindow(route, position):estimated_arrival = estimateArrivalTime(route, position)return TimeWindow(earliest=estimated_arrival - timedelta(minutes=5),latest=estimated_arrival + timedelta(minutes=5))

VRPTW with Duration Constraints (VRPTWDC) is applied for the full shift planning problem, incorporating flexible departure times and strict route duration limits (6-hour active shift maximum). Implementation via Google OR-Tools, which provides a production-grade VRPTW solver with support for time windows, capacity constraints, and duration limits.

3.4 Fatigue Mitigation Architecture

Energy Depletion Model

Rep energy is modeled as an exponential decay process with discrete recovery events (breaks). The biomathematical fatigue model integrates physical exertion, cognitive load, and emotional labor into a single energy state variable:

E(t) = E_max · e^(-kappat) + £recovery_events(t) where kappa = fatigue rate constant, E_max = 1.0 (normalized), recovery_events = +0.25 per 15-minute break

Recommended kappa = 0.08–0.12 per hour for field sales (calibrate against rep-reported fatigue

data). At kappa = 0.10: E(2h) = 0.819, E(4h) = 0.670, E(6h) = 0.549 before break recovery. With two 15-minute breaks at t=2h and t=4h: E(6h) approx 0.699 — a 27% improvement in end-of-shift energy state.

Three-Zone Intensity Framework

Table 3.4 — Three-Zone Fatigue Intensity Framework

Zone	Time Window	Energy State E(t)	Target Assignment	Routing Strategy	Conversion Rate Modifier
Zone 1 — Peak	0–2 hours	E > 0.80	Highest-propensity targets (S(p) > 0.70)	Maximum density; tightest cluster core	base_rate times 1.0 (full performance)
Zone 2 — Sustained	2–4 hours	0.60 < E leq 0.80	Medium-propensity targets (0.45–0.70)	Standard density; include break node at	base_rate times 0.85
Zone 3 — Conservation	4–6 hours	E leq 0.60	Easy wins + buffer targets (S(p) 0.35–0.55)	Reduce cluster density by 20%; pad with break	base_rate times 0.70

Rep Performance Degradation Curve: conversion_rate(t) = base_rate times (1 - fatigue_factor times t)

Where fatigue_factor = 0.04–0.06 per hour (empirically calibrated). At fatigue_factor = 0.05: conversion_rate(6h) = base_rate times 0.70 — a 30% degradation by end of shift without fatigue management. The three-zone framework, combined with break injection, targets limiting this degradation to leq15% by end of shift.

Adaptive Zone 3 Density Reduction

In Zone 3 (t > 4h), the routing engine automatically reduces cluster density by 20% by removing the lowest-propensity properties from the remaining route. This is implemented as a dynamic re-scoring step: at t=4h, re-rank remaining unvisited properties by S(p) and drop the bottom 20% from the active route. Dropped properties are logged for next-day reassignment, not permanently removed from the target pool.

3.5 Real-Time Route Adjustment Protocols

Real-time route adjustments are triggered by conversion outcomes, accessibility flags, and rep performance signals. The adjustment engine runs on the mobile app with a lightweight local solver, with full re-optimization available via API call to the routing service.

Table 3.5 — Real-Time Route Adjustment Trigger Matrix

Trigger Event	Detection Method	Adjustment Action	Re-optimization Scope
Positive conversion	Rep logs 'Converted' in app	Expand search radius by 15%; add 3 nearest neighbors to active route	Local: next 10 stops only
Negative streak (3+ consecutive rejections)	App detects 3 consecutive 'Not Interested' logs	Switch to adjacent cluster with highest R(C); re-run 2-Opt on new cluster	Full cluster re-optimization
Gated community / no-solicitation zone	Rep flags property; cross-reference no-solicitation database	Skip all properties in flagged zone; log for database update	Skip and continue
Pace deviation > 20% behind schedule	GPS tracking vs estimated pace	Trigger check-in notification; offer to drop bottom 10% of remaining stops	Selective pruning
Break compliance failure (break skipped)	Virtual node not checked in within time window	Alert rep; insert next break node 15 stops earlier	Break node repositioning
Property not accessible (locked gate, dog, etc.)	Rep flags 'Not Accessible'	Log flag; remove from today's route; schedule for different time/day	Remove and continue

Dynamic Re-Routing After Major Deviation

When a major route deviation occurs (cluster switch, large skip block, or significant pace deviation), the routing engine re-runs 2-Opt on the remaining unvisited properties in the current cluster. The re-optimization uses the rep's current GPS position as the new route start point, ensuring the updated route is optimal from the current location rather than the original start. Re-optimization latency target: <500ms for clusters leq 60 stops.

3.6 Morale Optimization Framework

Field sales performance is not purely a physical optimization problem — psychological momentum is a measurable performance variable. The morale optimization framework engineers the sequence of interactions to maximize psychological momentum throughout the shift.

Psychological Momentum Design Principles

- Post-Break Momentum Injection: The first 3 stops after each break are always the highest-propensity available targets (S(p) > 0.65). The psychological reset of a break,

combined with a high-quality target, maximizes conversion probability at the moment of peak re-engagement

- Consecutive High-Intensity Cap: No more than 8 consecutive high-propensity stops (S(p) > 0.70) before inserting a medium-propensity buffer stop (S(p) 0.45–0.65). Prevents emotional exhaustion from repeated high-stakes interactions
- Win-Rate Visibility: Real-time conversion rate displayed on app home screen vs daily target. Research confirms that visible progress toward goals increases persistence and effort. Display format: 'X conversions today — Y% of daily target'
- Cluster Completion Milestone: Visual celebration animation triggered every 10 stops completed. Provides dopamine-loop reinforcement for route completion behavior. Secondary milestone at cluster completion (full cluster finished)
- Early Win Sequencing: Within Zone 1, sequence the 2nd and 3rd stops as medium-propensity targets (not the absolute highest). This ensures the rep gets an early win before tackling the hardest targets, building confidence before peak difficulty

Morale optimization is not a soft feature — it is a conversion rate multiplier. A rep who has logged 2 conversions in the first hour approaches the next door differently than one who has logged zero. Engineering the sequence to maximize early wins and post-break momentum is a quantifiable performance lever that costs zero additional computational resources.

Section 4: Technical Implementation Blueprint

4.1 System Architecture Overview

Algorithm II is implemented as a five-stage pipeline with two processing modes: nightly batch (scoring, clustering, territory planning) and real-time on-demand (routing, feedback, dynamic adjustment). The architecture follows a microservices pattern with event-driven communication between services.

Five-Stage Pipeline

1. RentCast API Ingestion: Nightly batch pull of updated property records. Delta sync (only changed records) for efficiency. Raw payload stored in PostgreSQL + PostGIS. Estimated volume: 140M+ records total, ~50K–500K active target properties per deployment market
2. Feature Engineering Service: Transforms raw RentCast fields into normalized feature vectors. Computes f_decay(t), f_momentum, D(p), neighborhood context features. Outputs feature matrix to scoring queue. Runtime: ~2–4 hours for full market re-score
3. Score Engine: Applies weighted composite formula S(p) = £ w b f b (p) . Retrieves current Bayesian weights from weight store. Outputs calibrated propensity scores [0,1] to property database. Triggers cluster refresh flag if score delta > 0.15 for >10% of cluster
4. Clustering Selector: Reads density metrics for each deployment zone. Selects DBSCAN/HDBSCAN/K-Means per zone. Runs propensity-weighted clustering. Validates cluster quality metrics. Outputs cluster assignments to cluster store

5. Route Optimizer: On-demand (rep session start). Reads cluster assignment for rep's territory. Runs nearest-neighbor seed + 2-Opt refinement. Injects break nodes via VRPTW. Enforces loop closure. Returns route payload to mobile app in <2s

Microservices Architecture

Table 4.1 — Microservices Architecture Specification

Service	Responsibility	Processing Mode	Technology Stack	State Store
scoring-service	Feature engineering + propensity score computation	Nightly batch + on-demand re-score	Python, XGBoost, scikit-learn, Pandas	PostgreSQL (property scores)
clustering-service	Algorithm selection + weighted clustering +	Nightly batch + triggered refresh	Python, scikit-learn, HDBSCAN, PySAL	PostgreSQL + PostGIS (cluster assignments)
routing-service	TSP formulation + 2-Opt + break injection + loop closure	Real-time on-demand (<2s)	Python, Google OR-Tools, NumPy	Redis (active route state)
feedback-service	Conversion event ingestion + Bayesian weight update	Event-driven (real-time)	Python, FastAPI, SQLAlchemy	PostgreSQL (conversion events, weight store)
drift-monitor	PSI tracking + ADWIN drift detection + retraining triggers	Continuous background + weekly batch	Python, Evidently, Metaflow, River	PostgreSQL (drift metrics, model registry)

4.2 Algorithmic Pseudocode — All Three Engines

Engine 1: Scoring Engine

function scoringEngine(property_id):# Stage 1: Feature Extractionraw = rentcast_api.get_property(property_id)# Stage 2: Feature Engineeringt = years_since(raw.last_sale_date)lambda_val = getMarketLambda(raw.market_regime) # 0.10/0.15/0.20features = {'f_decay': 1 - exp(-lambda_val * t),'f_equity': minmax_normalize(raw.equity_percent, 0, 1),'f_vacancy': float(raw.is_vacant),'f_absentee': float(raw.absentee_owner) * log(1 + raw.owner_distance_km),'f_dom':

minmax_normalize(raw.days_on_market, 0, local_median_dom(raw.zip_code) * 3),'f_momentum': zscore_normalize(raw.current_value,rolling_mean_90d(raw.zip_code), rolling_std_90d(raw.zip_code)),'f_valuation_gap': max(0, raw.market_value - raw.assessed_value) / raw.market_value,'f_occupancy': -1.0 * float(raw.owner_occupied) # Inverse penalty}# Stage 3: Distress CompositeD = (0.35 * max(0, 0.70 - raw.equity_percent) / 0.70 +0.30 * features['f_vacancy'] +0.20 * min(1, raw.owner_distance_km / 100) +0.15 * features['f_dom']) features['f_distress'] = D# Stage 4: Clip all features to [0.01, 0.99]features = {k: clip(v, 0.01, 0.99) for k, v in features.items()}# Stage 5: Weighted Composite Scoreweights = bayesian_weight_store.get_current_weights()raw_score = sum(weights[k] * features[k] for k in features)# Stage 6: Distress Multiplierif D > 0.65:raw_score = min(0.99, raw_score * 1.15)# Stage 7: Calibration to Conversion Probabilitycalibrated_score = platt_scaling(raw_score, calibration_model.a,calibration_model.b)return PropertyScore(property_id=property_id, score=calibrated_score,features=features,distress_composite=D,timestamp=now())

Engine 2: Clustering Selector

function clusteringEngine(zone_id):properties = db.get_scored_properties(zone_id)bbox = computeBoundingBox(properties)density = len(properties) / bbox.area_km2# Algorithm Selectionif density > 150: # Urbank = optimalK_silhouette(properties, k_range=range(5, 26))clusterer = KMeans(n_clusters=k, init='k-means++', n_init=10,max_iter=300, tol=0.001)coords = [(p.lat, p.lon) for p in properties]weights = [p.score for p in properties]labels = clusterer.fit_predict(coords, sample_weight=weights)elif density > 30: # Suburban — HDBSCANmin_cs = max(8, int(len(properties) * 0.03))clusterer = HDBSCAN(min_cluster_size=min_cs,metric='haversine', cluster_selection_method='eom',allow_single_cluster=False)coords_rad = [(radians(p.lat), radians(p.lon)) for p in properties]labels = clusterer.fit_predict(coords_rad)else: # Rural — DBSCAN eps = adaptiveEpsilon(properties, k=3)clusterer = DBSCAN(eps=eps, min_samples=3, metric='haversine')coords_rad = [(radians(p.lat), radians(p.lon)) for p in properties]labels = clusterer.fit_predict(coords_rad)labels = handleNoisePoints(labels, properties) # Reassign high-value noise# Quality Validationsil_score = silhouette_score(coords, labels, metric='haversine') db_index = davies_bouldin_score(coords, labels)propensity_cv = computeIntraClusterCV(labels, [p.score for p in properties])if sil_score < 0.35 or db_index > 1.5 or propensity_cv > 0.40: log_quality_failure(zone_id, sil_score, db_index, propensity_cv)# Fallback: increase K by 2 and retry (K-Means) or adjust eps (DBSCAN)return clusteringEngine_fallback(zone_id, density, labels)# Revenue Density Scoringclusters = assignRevenueDensity(labels, properties)clusters = rankByRevenueDensity(clusters) # R(C) = sum(scores) / travel_timereturn ClusterAssignment(zone_id=zone_id, clusters=clusters,quality_metrics={'silhouette': sil_score, 'davies_bouldin': db_index,'propensity_cv': propensity_cv})

Engine 3: Routing Optimizer

function routingEngine(rep_id, cluster_id):cluster = db.get_cluster(cluster_id)properties = cluster.properties # 40-60 stopsrep_location = gps.getCurrentLocation(rep_id)# Stage 1: Nearest

Neighbor Seed Routeroute = nearestNeighborTSP(properties, start=rep_location)# Stage 2: 2-Opt Refinement (loop-constrained)route = twoOpt(route) # See Section 3.2 pseudocode# Stage 3: Enforce Loop Closureif route[0] != route[-1]:route = closeLoop(route) # Connect last stop back to start# Stage 4: Fatigue Zone Assignmentroute = assignFatigueZones(route) # Zone 1: stops 1-20, Zone 2: 21-40, Zone 3: 41+# Stage 5: Resequence for Morale Optimizationroute = moralOptimizeSequence(route)# - Move highest S(p) targets to Zone 1# - Ensure post-break stops are high-propensity# - Cap consecutive high-intensity stops at 8# Stage 6: Break Node Injection route = injectBreakNodes(route, interval=27, break_duration=15)# Stage 7: VRPTW Validation route = vrptw_validate(route,time_windows=computeTimeWindows(route),max_duration_hours=6, solver='google_or_tools')# Stage 8: Final Loop Closure Checkassert route[0].location == route[-1].location, 'Loop closure failed'return RoutePayload(rep_id=rep_id,cluster_id=cluster_id, stops=route,estimated_duration_hours=estimateDuration(route),break_positions=[i for i, s in enumerate(route) if s.node_type == 'MANDATORY_BREAK'], revenue_density=cluster.revenue_densitygenerated_at=now())

4.3 Mathematical Formulations Reference

Table 4.3 — Complete Mathematical Formulations Reference

Formula Name	Mathematical Notation	Variable Definitions	Parameter Ranges	Implementation Notes
Propensity Score	S(p) = £ bw b f b (p)	w b= feature weight, f b (p=) normalized feature value for property p	w bin [0,1], £ w b= 1.0, f bin [0.01, 0.99]	Apply Platt scaling post-computation for probability calibration
Ownership Decay	f_decay(t) = 1 - e^(-lambdat)	t = years since last_sale_date, lambda = decay	lambda = 0.10 (rural) to 0.25 (hot urban); default	Invert base decay: high t rightarrow high propensity
Market Momentum	f_momentum = (v - mu_90d) / sigma_90d	v = current market value, mu_90d = 90-day rolling mean, sigma_90d = 90-day rolling std	Clip to [-3, +3] sigma; normalize to [0,1]	Negative momentum = depreciation = distress signal (positive weight)
Distress Composite	D(p) = alpha·f_eq + beta·f_vac + gamma·f_dist + delta·f_dom	alpha=0.35, beta=0.30, gamma=0.20, delta=0.15; f_eq = equity gap, f_vac = vacancy, f_dist = absentee distance, f_dom = normalized DOM	D(p) in [0,1]; D > 0.65 triggers 1.15times multiplier	Parameters sum to 1.0; D is a valid sub-composite

Bayesian Update (alpha)	alpha_new = alpha_prior + conversions	alpha = Beta distribution success parameter, conversions = observed positive outcomes	alpha_prior = 3–5 (informed prior); grows with field data	Posterior mean = alpha/(alpha+beta)
Bayesian Update (beta)	beta_new = beta_prior + non_conversions	beta = Beta distribution failure parameter, non_conversions = observed negative outcomes	beta_prior = 15–19 (reflecting 15–20% baseline rate)	Posterior variance decreases as alpha+beta grows
Weighted Clustering	min £ b£,| i n Cvb,| · | | x,| m u b | | ²	w,|= S (p,╞) propensity score, x,|= coordinates, m u ╞ weighted centroid	w,|in [0,1]; use sklearn sample_weight parameter	High-propensity points pull centroids toward them
Silhouette Score	s(i) = [b(i) - a(i)] / max(a(i), b(i))	a(i) = mean intra-cluster distance, b(i) = mean nearest-cluster distance	s(i) in [-1, 1]; target avg geq 0.35	Use haversine metric for geographic coordinates
Revenue Score	R(C) = £,| i n C S (p,ƒ) travel_time(C)	S (p,╞)propensity score of property j, travel_time(C) = estimated traversal time in hours	R(C) > 0; rank clusters descending	Primary cluster prioritization metric
Revenue Density	rho(C) = £,| i n C S (p,ƒ)area(C)	area(C) = cluster bounding area in km²	rho(C) > 0; higher = more concentrated value	Secondary metric for geographic concentration
Loop Efficiency	E_loop = (D_line - D_loop) / D_line	D_line = linear route distance, D_loop = loop route distance	Expected E_loop = 0.15–0.25	Measure at cluster level; report in routing dashboard
Energy Depletion	E(t) = E_max · e^(-kappat) + £ recovery	E_max = 1.0, kappa = fatigue rate, t = hours elapsed, recovery = +0.25 per break	kappa = 0.08–0.12/hour; default kappa = 0.10	Zone thresholds: Zone 1 E>0.80, Zone 2 0.60<Eleq0.80, Zone 3 Eleq0.60

Conversion Degradation	CR(t) = base_rate times (1 - fatigue_factor times t)	base_rate = rep's historical conversion rate, fatigue_factor = degradation per hour	fatigue_factor = 0.04–0.06/hour	Use to weight target sequencing by zone

4.4 firstknock.online Integration Architecture

API Layer — REST Endpoints

Table 4.4 — REST API Endpoint Specification

Endpoint	Method	Service	Processing Mode	Response SLA	Description
POST /score/{property_id}	POST	scoring-service	On-demand	< 200ms	Re-score single property with current weights
POST /score/batch	POST	scoring-service	Async batch	< 30min for 100K	Batch re-score for nightly pipeline
GET /cluster/{zone_id}	GET	clustering-service	Cached (nightly)	< 100ms	Retrieve current cluster assignments for zone
POST /cluster/refresh/{zone_id}	POST	clustering-service	Async	< 5min	Trigger cluster refresh for zone
POST /route	POST	routing-service	Real-time	< 2s	Generate optimized route for rep session
POST /route/adjust	POST	routing-service	Real-time	< 500ms	Dynamic route adjustment after deviation
POST /feedback/conversion	POST	feedback-service	Event-driven	< 50ms	Log conversion outcome; trigger weight update

GET /weights/current	GET	feedback-service	Cached	< 50ms	Retrieve current Bayesian weight configuration
GET /metrics/drift	GET	drift-monitor	Cached (hourly)	< 100ms	Retrieve current PSI and drift status for all features

Data Flow Architecture

The data flow follows a unidirectional pipeline with a feedback loop: (1) RentCast webhook fires nightly at 01:00 with updated property records rightarrow (2) scoring-service consumes webhook payload, runs feature engineering, writes scores to PostgreSQL rightarrow (3) clustering-service reads updated scores, detects zones with >10% score delta, triggers cluster refresh for affected zones rightarrow (4) cluster assignments written to PostGIS with geographic boundaries rightarrow (5) Rep opens mobile app, requests route rightarrow (6) routing-service reads cluster assignment, runs TSP + 2-Opt + break injection, returns route payload in <2s rightarrow (7) Rep works route, logs conversion outcomes rightarrow (8) feedback-service receives conversion events, updates Beta(alpha,beta) parameters rightarrow (9) drift-monitor runs PSI check every 4 hours, flags anomalies rightarrow (10) If PSI > 0.20 or AUC < 0.70, retraining job queued rightarrow cycle repeats.

Mobile App Data Contract

Route Payload Schema (JSON):

{"route_id": "uuid","rep_id": "uuid","cluster_id": "uuid","generated_at": "ISO8601", "estimated_duration_hours": 5.5,"revenue_density": 0.847,"stops": [{"stop_id": "uuid","property_id": "rentcast_id | null (break node)","node_type": "TARGET | MANDATORY_BREAK","sequence": 1, "coordinates": {"lat": 33.4484, "lon": -112.0740},"propensity_score": 0.82,"distress_composite": 0.71,"fatigue_zone": 1,"estimated_arrival": "ISO8601","break_duration_min": null,"top_signals": ["high_equity", "absentee_distant", "long_tenure"]}],"break_positions": [27, 54],"loop_closure": true}

Conversion Event Schema (JSON):

{"event_id": "uuid","rep_id": "uuid","property_id": "rentcast_id","route_id": "uuid","weight_variant_id": "uuid","timestamp": "ISO8601","fatigue_zone": 1,"outcome": "CONVERTED | INTERESTED | NOT_INTERESTED | NOT_HOME | NOT_ACCESSIBLE","interaction_duration_sec": 180,"notes": "optional free text"}

4.5 Performance Benchmarking Criteria

Table 4.5 — Performance Benchmarking Criteria: Algorithm I Baseline vs Algorithm II

Targets

KPI	Algorithm I Baseline	Algorithm II Target	Measurement Method	Review Cadence
Conversions per shift	Baseline (current avg)	+25–40% improvement	feedback-service conversion count per rep session	Weekly
Revenue density per cluster rho(C)	Baseline (current avg)	+20–35% improvement	£ S (p,I) area(C) computed post-cluster assignment	Weekly
Route distance per conversion	Baseline (current avg)	" 1 5 – 2 5 % reduction	GPS total distance / conversions logged per session	Weekly
AUC-ROC	~0.60 (estimated)	geq 0.75	Weekly validation job on temporal holdout	Weekly
Brier Score	Not tracked	leq 0.15	Weekly calibration plot generation	Weekly
Cluster Silhouette Score	Not tracked	geq 0.35	Computed at cluster formation; logged to metrics DB	Nightly
Route generation latency	Not tracked	< 2s for leq 60 stops	routing-service response time monitoring	Continuous
Break compliance rate	Not tracked	geq 85%	Break node check-in rate from mobile app events	Daily
Score calibration (PSI)	Not tracked	PSI < 0.20 (stable)	drift-monitor PSI computation	Every 4 hours
Weight drift detection latency	Not tracked	< 48 hours	Time from drift onset to ADWIN alert	Continuous

4.6 KPI Framework

Primary KPIs — Business Performance

- Conversion Rate Per Shift: Total conversions / total doors knocked per rep session. Primary measure of Algorithm II's targeting quality. Target: +25–40% vs Algorithm I baseline
- Revenue Density Per Cluster: rho(C) = £ S (p,I) area(C). Measures how efficiently the clustering algorithm concentrates high-value targets. Target: +20–35% improvement
- Route Completion Rate: Percentage of planned stops actually visited per session. Measures routing quality and fatigue management effectiveness. Target: geq 90%

Secondary KPIs — Model Quality

- Score Calibration Accuracy: AUC-ROC geq 0.75, Brier Score leq 0.15, Precision-Recall AUC geq 0.25
- Cluster Silhouette Score: Average geq 0.35 across all active clusters. Tracks geographic clustering quality
- Break Compliance Rate: geq 85% of scheduled breaks taken within time window. Tracks fatigue management adherence
- Weight Variant Performance: Thompson Sampling posterior mean for winning variant vs baseline. Tracks Bayesian calibration effectiveness

Leading Indicators — Early Warning System

- Propensity Score Distribution Shift: PSI > 0.15 on any primary feature signals upcoming model degradation — investigate before performance drops
- Cluster Density Trends: Declining average cluster propensity density (mean S(p) per cluster) signals market condition change — trigger regime detection review
- Conversion Streak Patterns: Increasing negative streak frequency (3+ consecutive rejections) signals either targeting quality degradation or market regime shift

Lagging Indicators — Trend Confirmation

- 30-Day Conversion Rate Trend: Rolling 30-day conversion rate vs 90-day baseline. Confirms whether leading indicator alerts represent genuine degradation or noise
- Rep Retention Rate: Field rep retention as a function of route quality and fatigue management. Algorithm II's fatigue architecture should improve rep retention by reducing physical burnout

Section 5: Implementation Roadmap

The implementation roadmap is structured as four sequential phases over 16 weeks, each building on the previous. Phases are designed to deliver incremental value — Phase 1 alone produces a measurably better scoring model, even before clustering and routing upgrades are complete. Each phase includes explicit success criteria and rollback conditions to ensure production stability throughout the migration.

Objectives

Replace Algorithm I's binary flag extraction with the full continuous feature engineering pipeline. Implement all features defined in Section 1.3. Establish the normalization pipeline and initial weight configuration. Validate scoring quality on historical holdout data before any production deployment.

Task Breakdown

1. Week 1: RentCast field mapping — implement raw field rightarrow engineered feature transformation for all 12 primary fields. Build normalization pipeline (min-max, z-score, sigmoid). Unit test each feature transformation against known property records
2. Week 2: Decay function implementation — f_decay(t) with configurable lambda parameter. Market momentum feature f_momentum with 90-day rolling window. Neighborhood context enrichment (absorption rate, vacancy density, permit velocity) via spatial join in PostGIS
3. Week 3: Distress composite D(p) implementation. Life event proxy feature engineering. Full feature matrix assembly and validation. Verify feature distributions match expected ranges on 10K sample properties
4. Week 4: Initial weight configuration deployment (Table 1.2 values). Score all target properties in staging environment. Run AUC-ROC, Brier Score, and Lift Curve evaluation on temporal holdout. Compare against Algorithm I baseline scores

Table 5.1 — Phase 1 Success Criteria and Rollback Conditions

Criterion	Target	Measurement	Rollback Condition
Feature pipeline coverage	100% of target properties scored	Count of NULL scores in staging DB	Any feature producing >5% NULL rate rightarrow debug before proceeding
AUC-ROC on holdout	geq 0.70	Weekly validation job output	AUC < 0.60 after 2-week validation window rightarrow revert to Algorithm I
Brier Score	leq 0.20 (Phase 1 target)	Calibration plot generation	Brier > 0.25 rightarrow apply Platt scaling recalibration before Phase 2
Feature distribution validation	All features within expected ranges	Distribution plots vs reference	Any feature with >10% out-of-range values rightarrow investigate data quality

Objectives

Implement the three-tier density classifier and HDBSCAN integration. Deploy propensity-weighted K-Means for urban zones. Implement the full cluster quality validation suite. Establish revenue density scoring and cluster prioritization.

Task Breakdown

1. Week 5: Density classifier implementation — compute property density per deployment zone, implement URBAN/SUBURBAN/RURAL threshold logic. Calibrate thresholds against 20 representative zones (see Section 2.1 calibration methodology)
2. Week 6: HDBSCAN integration with Haversine metric. Adaptive epsilon calculation for DBSCAN (k-distance graph + Kneedle algorithm). Noise point handling logic (high-propensity reassignment vs low-propensity flagging)
3. Week 7: Propensity-weighted K-Means deployment (sklearn sample_weight). DBSCAN density weighting via point replication. Revenue density scoring R(C) and rho(C) computation. Cluster prioritization ranking
4. Week 8: Full quality validation suite (silhouette, Davies-Bouldin, intra-cluster CV, propensity density). REDCAP contiguity enforcement via PySAL. A/B comparison of Algorithm II clusters vs Algorithm I clusters on same zones

Table 5.2 — Phase 2 Success Criteria and Rollback Conditions

Criterion	Target	Measurement	Rollback Condition
Silhouette Score	geq 0.35 average across all zones	Cluster quality validation suite	Silhouette < 0.25 for >50% of zones rightarrow revert to Algorithm I clustering
Cluster propensity density improvement	geq 15% vs Algorithm I baseline	Mean S(p) per cluster: Algorithm II vs Algorithm I	No improvement after 3 consecutive days rightarrow investigate weight integration
Cluster size compliance	90% of clusters within 40–60 stops	Count of out-of-range clusters	Persistent out-of-range clusters rightarrow adjust K selection constraints
Revenue density improvement	rho(C) geq 10% above Algorithm I	rho(C) comparison across matched zones	No improvement rightarrow investigate propensity-weighted clustering implementation

Objectives

Implement loop-constrained TSP formulation. Deploy 2-Opt with loop closure enforcement. Integrate virtual break node injection via VRPTW. Implement three-zone fatigue routing and morale optimization sequence.

Task Breakdown

1. Week 9: Loop-constrained TSP formulation — modify nearest-neighbor construction to enforce Hamiltonian cycle. Implement loop closure check and correction. Measure E_loop vs Algorithm I linear routes on 50 representative clusters
2. Week 10: 2-Opt refinement with O(1) symmetric length delta. Or-Opt fallback for n > 200. Convergence criteria implementation (” d i s t a n c e < 0.001 or max_iter=1000). Performance benchmark: route generation < 2s for 60 stops
3. Week 11: Virtual break node injection (VRPTW formulation via Google OR-Tools). Break location finder (nearest parking/shade/convenience store via Google Places API). Time window constraint enforcement. Break compliance tracking in mobile app
4. Week 12: Three-zone fatigue routing (Zone 1/2/3 target sequencing). Morale optimization sequence (post-break high-propensity injection, consecutive cap at 8). Real-time route adjustment protocols (conversion triggers, accessibility flags). Pilot deployment with 5 reps for 2-week field validation

Table 5.3 — Phase 3 Success Criteria and Rollback Conditions

Criterion	Target	Measurement	Rollback Condition
Route distance per conversion	geq 10% reduction vs Algorithm I	GPS distance / conversions per session	No improvement after 2-week pilot rightarrow investigate loop topology implementation
Break compliance rate	geq 85%	Break node check-in rate from mobile app	< 70% compliance rightarrow investigate break location quality and time window
Route generation latency	< 2s for leq 60 stops	routing-service response time monitoring	> 5s consistently rightarrow optimize 2-Opt implementation or reduce cluster size

Rep route completion rate	geq 90%	Stops visited / stops planned per session	Completion rate drops > 10% vs Algorithm I rightarrow investigate cluster size and fatigue zone calibration

Phase 4: Feedback Loop & Continuous Calibration (Weeks 13–16)

Objectives

Deploy the Bayesian weight update pipeline. Implement Thompson Sampling A/B framework for weight variant testing. Establish drift detection monitoring (PSI, ADWIN, Page-Hinkley). Configure weekly batch recalibration jobs and performance-based retraining triggers.

Task Breakdown

1. Week 13: Bayesian weight update pipeline — feedback-service consuming conversion events, updating Beta(alpha,beta) parameters every 4 hours. Weight store implementation in PostgreSQL. Posterior mean and variance tracking dashboard
2. Week 14: Thompson Sampling A/B framework — 3 initial weight variants (Algorithm I weights, Algorithm II initial weights, momentum-heavy variant). Variant assignment logic with epsilon=0.10 exploration rate. Deconfounded Thompson Sampling (DTS) for macro regime events
3. Week 15: Drift detection deployment — Evidently PSI monitoring for all 12 primary features. ADWIN concept drift detection on rolling AUC. Page-Hinkley secondary confirmation. Alert routing to engineering Slack channel. KL divergence threshold monitoring
4. Week 16: Weekly recalibration job automation via Metaflow. Performance-based retraining triggers (AUC < 0.70, PSI > 0.25, Brier > 0.20). Full Algorithm II production deployment across all active markets. 30-day post-deployment monitoring period

Table 5.4 — Phase 4 Success Criteria and Rollback Conditions

Criterion	Target	Measurement	Rollback Condition
Weight drift detection latency	< 48 hours from drift onset to alert	Time between PSI threshold breach and alert generation	Latency > 72h rightarrow investigate monitoring pipeline
Calibration Brier Score	leq 0.15	Weekly validation job	Brier > 0.20 after recalibration rightarrow trigger full retraining

Thompson Sampling convergence	Winning variant identified within 500 interactions	Posterior mean separation between variants	No variant separation after 1000 interactions rightarrow investigate variant design
Score distribution stability	KL divergence < 0.15 vs baseline	drift-monitor KL computation	KL > 0.30 without recovery in 72h rightarrow automatic rollback to last validated checkpoint

Appendix: Complete Mathematical Reference

This appendix consolidates all mathematical formulations introduced throughout the specification into a single reference table. All formulas are production-grade recommendations derived from peer-reviewed PropTech research. Parameter ranges represent empirically validated bounds; default values are recommended for initial deployment.

Appendix Table A.1 — Complete Mathematical Formulations Reference: All Formulas, Parameters, and Implementation Notes

Formula Name	Notation	Variable Definitions	Parameter Ranges	Default Values	Implementation Notes
Composite Propensity Score	S(p) = £ bw b f b (p)	w bfeature weight; f b (p) : normalized feature for property p; i in {decay, equity, vacancy, absentee, momentum, distress, dom, valuation_gap}	w bin [0,1]; £ w b= 1.0; f b in [0.01, 0.99]	See Table 1.2	Apply Platt scaling post-computation; output in [0,1] as conversion probability
Ownership Duration Decay	f_decay(t) = 1 - e^(-lambdat)	t: years since last_sale_date; lambda: market-calibrated decay constant	lambda in [0.10, 0.25]; t in [0, 50]	lambda = 0.15	Invert base decay so high t rightarrow high propensity; regime-adjust lambda

Market Momentum	f_momentum = (v - mu_90d) / sigma_90d	v: current market value; mu_90d: 90-day rolling mean; sigma_90d: 90-day rolling std	Clip to [-3, +3]sigma; normalize to [0,1]	90-day window	Negative momentum = depreciation = distress signal
Distress Composite	D(p) = alpha·f_eq + beta·f_vac + gamma·f_dist + delta·f_dom	alpha: equity gap weight; beta: vacancy weight; gamma: absentee distance weight; delta: DOM weight;	alpha+beta+gamma+delta = 1.0; D(p) in [0,1]	alpha=0.35, beta=0.30, gamma=0.20, delta=0.15	D > 0.65 triggers 1.15times score multiplier, capped at 0.99
Distress Multiplier	S_final = min(0.99, S(p) times 1.15) if D(p) > 0.65	S(p): base composite score; D(p): distress composite	Multiplier = 1.15; cap = 0.99	Threshold = 0.65	Applied after weighted sum, before Platt scaling
Bayesian Prior (Beta)	theta ~ Beta(alpha, beta)	alpha: success parameter (conversions); beta: failure parameter (non-conversio	alpha_prior = 3–5; beta_prior = 15–19	Beta(3, 17)	Reflects ~15% baseline conversion rate prior
Bayesian Update (alpha)	alpha_new = alpha_prior + conversions	conversions: observed positive outcomes in field data	Grows unbounded with data	Start: alpha = 3	Posterior mean = alpha/(alpha+beta)
Bayesian Update (beta)	beta_new = beta_prior + non_conversions	non_conversions: observed negative outcomes in field data	Grows unbounded with data	Start: beta = 17	Posterior variance rightarrow 0 as alpha+beta rightarrow
Posterior Mean	mu_post = alpha_new / (alpha_new + beta_new)	alpha_new, beta_new: updated Beta parameters	mu_post in [0,1]	~0.15 initially	Use as current best estimate of conversion probability

Posterior Variance	sigma²_post = (alpha·beta) / ((alpha+beta)² .	alpha, beta: current Beta parameters	sigma²_post rightarrow 0 as sample size grows	High initially	Width of confidence interval around posterior mean
Weighted K-Means Objective	min £ b£,| i n C b w,|· | | x,-| m u b | | ²	w,| propensity score S (p,|) ; x,| : coordinates; m u b weighted centroid of cluster i	w,|in [0,1]	w,|= S (p,|)	Pass as sample_weight to sklearn KMeans
Silhouette Score	s(i) = [b(i) - a(i)] / max(a(i), b(i))	a(i): mean intra-cluster distance; b(i): mean nearest-cluster distance for point i	s(i) in [-1, 1]; avg geq 0.35 required	Target: 0.35–0.60	Use haversine metric for lat/lon coordinates
Revenue Score	R(C) = £,| i n C S (p,|) travel_time(C)	S (p,|) : propensity score; travel_time(C): estimated traversal time in hours	R(C) > 0; rank descending	Varies by cluster	Primary cluster prioritization metric for rep assignment
Revenue Density	rho(C) = £,| i n C: S (p,|) area(C)	area(C): cluster bounding area in km²	rho(C) > 0	Varies by market	Geographic concentration metric; track improvement vs Algorithm I
Loop Efficiency Gain	E_loop = (D_line - D_loop) / D_line	D_line: linear route distance; D_loop: loop route distance	E_loop in [0.10, 0.35]	Expected: 0.15–0.25	Measure at cluster level; report in routing dashboard
Energy Depletion	E(t) = E_max · e^(-kappat) + £recovery	E_max = 1.0; kappa: fatigue rate; t: hours elapsed; recovery = +0.25 per 15-min break	kappa in [0.08, 0.12]/hour	kappa = 0.10	Zone thresholds: Z1 E>0.80, Z2 0.60<Eleq0.80, Z3 Eleq0.60

Conversion Degradation	CR(t) = base_rate times (1 - fatigue_factor times t)	base_rate: rep historical conversion rate; fatigue_factor: degradation per hour; t: hours elapsed	fatigue_factor in [0.04, 0.06]/hour	0.05/hour	Use to weight target sequencing by fatigue zone
PSI (Feature Drift)	PSI = £(A% - E%) times ln(A% / E%)	A%: actual bin proportion; E%: expected (baseline) bin proportion	PSI < 0.10: stable; 0.10–0.20: monitor; > 0.20: alert; > 0.25: retrain	Alert: 0.20	Compute weekly for all 12 primary features
KL Divergence (Score Drift)	KL(P||Q) = £ P(x) · ln(P(x)/Q(x))	P: current score distribution; Q: baseline score distribution	KL < 0.15: stable; > 0.30: rollback	Alert: 0.15	Compute on full score distribution; triggers rollback at 0.30
Absorption Rate	AR = properties_sold_30d / active_listings	properties_sold_30d: sales in last 30 days; active_listings: current inventory	AR < 0.15: buyer's market; 0.15–0.20: balanced; > 0.20: seller's market	Varies by market	Used as market regime feature and weight modulator

Technology Stack Summary

Appendix Table A.2 — Complete Technology Stack Reference

Component	Technology	Version / Notes	Purpose
Gradient Boosting	XGBoost	Latest stable	Primary propensity scoring model; probabilistic output
Clustering	scikit-learn	HDBSCAN, KMeans, DBSCAN modules	All three clustering algorithms; silhouette/Davies-Bouldi
Spatial Statistics	PySAL	libpysal + esda modules	Getis-Ord Gi* hotspot analysis; REDCAP contiguous zoning

Routing Optimization	Google OR-Tools	VRPTW solver	Break node injection; time window constraints; loop closure
Drift Monitoring	Evidently	DataDriftPreset + ModelPerformancePreset	PSI computation; feature drift dashboards; automated alerts
Pipeline Orchestration	Metaflow	AWS/local deployment	Nightly batch pipeline; retraining job scheduling
Experiment Tracking	MLflow	Model registry integration	Weight variant tracking; model versioning; rollback management
Streaming Drift	River (scikit-multiflow)	ADWIN + Page-Hinkley	Real-time concept drift detection on rolling AUC
Geospatial Database	PostgreSQL + PostGIS	PostGIS 3.x	Property storage; spatial joins; cluster boundary storage
Real-Time State	Redis	Redis 7.x	Active route state; rep session management; real-time score cache
API Framework	FastAPI	Python 3.11+	REST endpoints for all five microservices
Numeric Computing	NumPy / SciPy	Latest stable	Decay functions; distance calculations; statistical tests

Key Risks & Mitigations

Appendix Table A.3 — Risk Register with Mitigations and Monitoring Signals

Risk	Probability	Impact	Mitigation Strategy	Monitoring Signal
Concept drift from market volatility (interest rate shifts, seasonal cycles)	High	High	Evidently MLOps pipeline with ADWIN + PSI monitoring; performance-based retraining triggers (AUC < 0.70)	PSI > 0.20 on any primary feature

Computational bottleneck on 140M+ property records	Medium	Medium	Mini-batch K-Means (batch_size=1024); O(1) symmetric 2-Opt evaluations; delta-sync RentCast ingestion	Nightly batch job duration > 6 hours
Class imbalance degrading model performance (1–5% conversion rate)	High	High	Precision-Recall AUC as primary metric; SMOTE oversampling in training; temporal holdout validation	Precision-Recall AUC < 0.20
Bayesian prior misspecification causing slow calibration	Medium	Medium	Informed priors from Algorithm I conversion data; Thompson Sampling exploration rate epsilon=0.10 for rapid variant	Posterior mean not converging after 500 interactions
HDBSCAN producing too many noise points in sparse markets	Medium	Low	Noise point handling logic (high-propensity reassignment); fallback to DBSCAN with adaptive epsilon	Noise ratio > 25% of zone properties
Rep fatigue model miscalibration (kappa too high/low)	Low	Medium	Calibrate kappa against rep-reported fatigue data in pilot; adjust per rep cohort; break compliance rate	Break compliance rate < 70%
Score distribution divergence post-deployment	Medium	High	KL divergence monitoring with automatic rollback at KL > 0.30; checkpoint management in MLflow	KL > 0.15 on score distribution

Algorithm II: Predictive Propensity — Engineering Specification v2.0 | FirstKnock Sales OS |

This document is the authoritative engineering blueprint for Algorithm II. All formulas, parameter ranges, pseudocode, and architectural recommendations are derived from peer-reviewed PropTech research and are intended for direct implementation by the FirstKnock development team. Questions and implementation feedback should be directed to the FirstKnock Engineering team.